

# Containers e.g., docker

- Light(-er) weight
  - Allows them to be easily distributed
  - Rather than virtualizing the entire OS, it continues to use the host's **kernel**/operating system as a “base” to service whatever is running within the container.
- Faster than VMs (usually)
  - Can also use an emulated system if necessary → runs within a VM
- Designed for ephemerality
  - Containers are “disposable” – any long-term data should be stored in separate persistent “volumes”

# Containerization (in general)

You can take an application and wrap it in a container to ensure a consistent running environment.

- You can define the **operating system** it expects to use (**but not the kernel**)
- You can define the **CPU architecture** the program expects (and if the CPU architecture differs from the host, it will have to run within a virtual machine)
- You can define dependencies and other programs that the application expects to be installed
- You can define the “hard drive” layout the program expects
- All this, and the application gets **a level of isolation** from other applications on the system.

# docker

**docker** is a popular tool to create and manage containers

Operational components:

- **Containers** are an ephemeral object representing a copy of a program, based on an Image
- **Images** are built from **Dockerfiles**, and represent a frozen copy of an application and everything needed to run it
- **Dockerfiles** are special scripts that are used to build images, including:
  - Instructions on adding files (e.g. program files) from your local system
  - Instructions on adding dependencies
- **Volumes** store persistent data even past the lifetime of a container.

# docker

**docker** intuition:

- **Containers** are like the downloaded application
- **Images** are like the .zip, .msi, or .dmg that you download from the website
- **Dockerfiles** are like scripts that create the .zip/.msi/.dmg
- **Volumes** are like the places on your computer where your applications store data

# **docker**

Let's run a basic ubuntu system using Docker!

# Demo: run Ubuntu using docker

Let's run a basic ubuntu system using Docker!

```
docker run -it ubuntu:latest bash
```

# Demo: run Ubuntu using docker

Let's run a basic ubuntu system using Docker!

```
docker run -it ubuntu:latest bash
```

runs a new container based on an image

# Demo: run Ubuntu using docker

Let's run a basic ubuntu system using Docker!

```
docker run -it ubuntu:latest bash
```

Keep **STDIN** open (allows us to write things into the container)



# Demo: run Ubuntu using docker

Let's run a basic ubuntu system using Docker!

```
docker run -it ubuntu:latest bash
```

Allocate a **TTY** to the container (allows the container to receive things we write)

# Demo: run Ubuntu using docker

Let's run a basic ubuntu system using Docker!

```
docker run -it ubuntu:latest bash
```

All together: **“Run interactively”**

# Demo: run Ubuntu using docker

Let's run a basic ubuntu system using Docker!

```
docker run -it ubuntu:latest bash
```

Specifies the **image** to run (the **latest** version of **ubuntu**)

# Demo: run Ubuntu using docker

Let's run a basic ubuntu system using Docker!

```
docker run -it ubuntu:latest bash
```

Specifies which command to run within the image (**bash**)

# Dockerfiles

**Dockerfiles**, are script files which always have the name **Dockerfile**. They have the following syntax:

```
FROM ubuntu:latest

RUN apt-get -y update
RUN apt-get -y upgrade
RUN apt-get install -y build-essential
WORKDIR /hello_world
ADD ./hello_world/ ./
RUN make clean; make
ENTRYPOINT ["./hello_world"]
```

# Dockerfiles

**Dockerfiles**, are script files which always have the name **Dockerfile**. They have the following syntax:

**FROM** specifies the base of the image you're building.

```
FROM ubuntu:latest

RUN apt-get -y update
RUN apt-get -y upgrade
RUN apt-get install -y build-essential
WORKDIR /hello_world
ADD ./hello_world/ ./
RUN make clean; make
ENTRYPOINT ["./hello_world"]
```

# Dockerfiles

**Dockerfiles**, are script files which always have the name **Dockerfile**. They have the following syntax:

**FROM** specifies the base of the image you're building.

In this case, we're basing our image off the **latest** version of **ubuntu**.

```
FROM ubuntu:latest
```

```
RUN apt-get -y update
RUN apt-get -y upgrade
RUN apt-get install -y build-essential
WORKDIR /hello_world
ADD ./hello_world/ ./
RUN make clean; make
ENTRYPOINT ["./hello_world"]
```

# Dockerfiles

**Dockerfiles**, are script files which always have the name **Dockerfile**. They have the following syntax:

**RUN commands will run the given command inside a shell.**

```
FROM ubuntu:latest

RUN apt-get -y update
RUN apt-get -y upgrade
RUN apt-get install -y build-essential
WORKDIR /hello_world
ADD ./hello_world/ ./
RUN make clean; make
ENTRYPOINT ["./hello_world"]
```



# Dockerfiles

**Dockerfiles**, are script files which always have the name `Dockerfile`.

They have the following syntax:

**RUN commands will run the given command inside a shell.**

These commands form **intermediate containers**; each command builds off from the previous.

```
FROM ubuntu:latest

RUN apt-get -y update
RUN apt-get -y upgrade
RUN apt-get install -y build-essential
WORKDIR /hello_world
ADD ./hello_world/ ./
RUN make clean; make
ENTRYPOINT ["./hello_world"]
```

# Dockerfiles

**Dockerfiles**, are script files which always have the name **Dockerfile**. They have the following syntax:

**WORKDIR** specifies a new working directory from that point onwards. (it's like a **cd** that sticks)

```
FROM ubuntu:latest

RUN apt-get -y update
RUN apt-get -y upgrade
RUN apt-get install -y build-essential
WORKDIR /hello_world
ADD ./hello_world/ ./
RUN make clean; make
ENTRYPOINT ["./hello_world"]
```

# Dockerfiles

**Dockerfiles**, are script files which always have the name `Dockerfile`.

They have the following syntax:

**ADD will copy files from your local directory (relative to where the Dockerfile is located) into the image.**

```
FROM ubuntu:latest

RUN apt-get -y update
RUN apt-get -y upgrade
RUN apt-get install -y build-essential
WORKDIR /hello_world
ADD ./hello_world/ ./
RUN make clean; make
ENTRYPOINT ["./hello_world"]
```

# Dockerfiles

**Dockerfiles**, are script files which always have the name **Dockerfile**. They have the following syntax:

We can run **make** at this point because we installed it earlier.

```
FROM ubuntu:latest

RUN apt-get -y update
RUN apt-get -y upgrade
RUN apt-get install -y build-essential
WORKDIR /hello_world
ADD ./hello_world/ ./
RUN make clean; make
ENTRYPOINT ["./hello_world"]
```

# Dockerfiles

**Dockerfiles**, are script files which always have the name **Dockerfile**. They have the following syntax:

**ENTRYPOINT** describes what should happen when we run the image.

```
FROM ubuntu:latest

RUN apt-get -y update
RUN apt-get -y upgrade
RUN apt-get install -y build-essential
WORKDIR /hello_world
ADD ./hello_world/ ./
RUN make clean; make
ENTRYPOINT ["./hello_world"]
```

# Dockerfiles

**Dockerfiles**, are script files which always have the name `Dockerfile`.

They have the following syntax:

**ENTRYPOINT** describes what should happen when we run the image.

Here it says we should run the `./hello_world` program, which we just built

```
FROM ubuntu:latest

RUN apt-get -y update
RUN apt-get -y upgrade
RUN apt-get install -y build-essential
WORKDIR /hello_world
ADD ./hello_world/ ./
RUN make clean; make
ENTRYPOINT ["./hello_world"]
```

# Dockerfiles

**Dockerfiles**, are script files which always have the name **Dockerfile**. They have the following syntax:

At the end, **docker** will combine all the intermediate containers into a static **image**. Each command forms a **layer** of that image.

```
FROM ubuntu:latest

RUN apt-get -y update
RUN apt-get -y upgrade
RUN apt-get install -y build-essential
WORKDIR /hello_world
ADD ./hello_world/ ./
RUN make clean; make
ENTRYPOINT ["./hello_world"]
```